Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

Hardware-Software-Systeme

Übung 02: Digitaler Entwurf | Sequentielle Logik

M.Sc. E. Trumann, M.Sc. J. Homann, Prof. Dr.-Ing. G. Payá Vayá,
1. November 2024

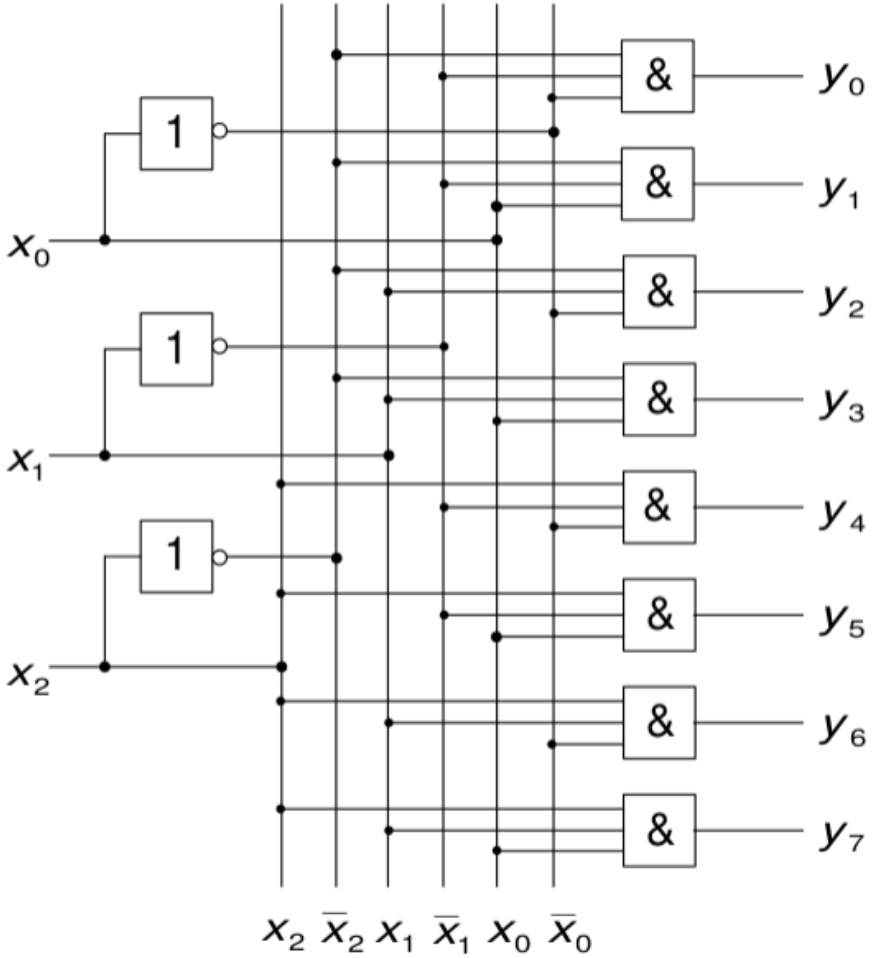
Gliederung

- **Multiplexer**
  - Codeumsetzer für 1-aus-n Codes
  - Multiplexer und Demultiplexer
- **Sequentielle Schaltungen**
  - Speicher
  - Flip-Flop-Typen
  - Taktung von Flip-Flops
  - Register
- **Übungen**
  - Multiplexer
  - Demultiplexer
  - Multiplexer-Schaltung
- **EDA Playground und GIT-Übungen**

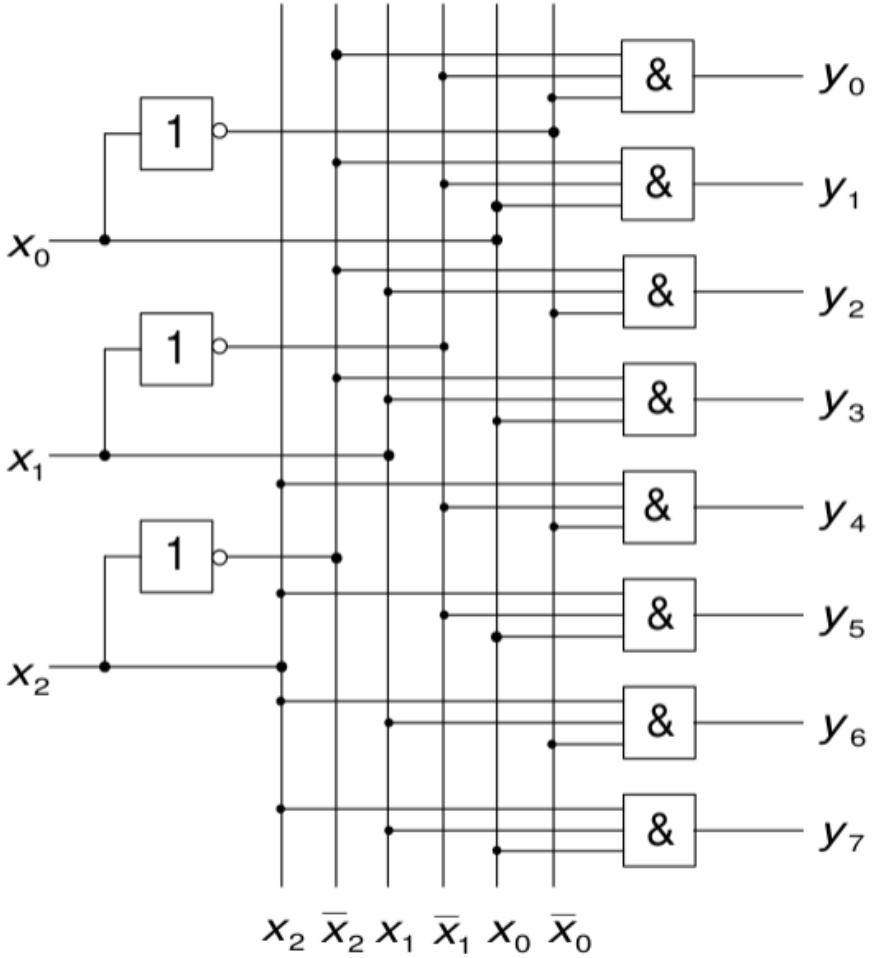
(One-Hot-)Decoderschaltnetz

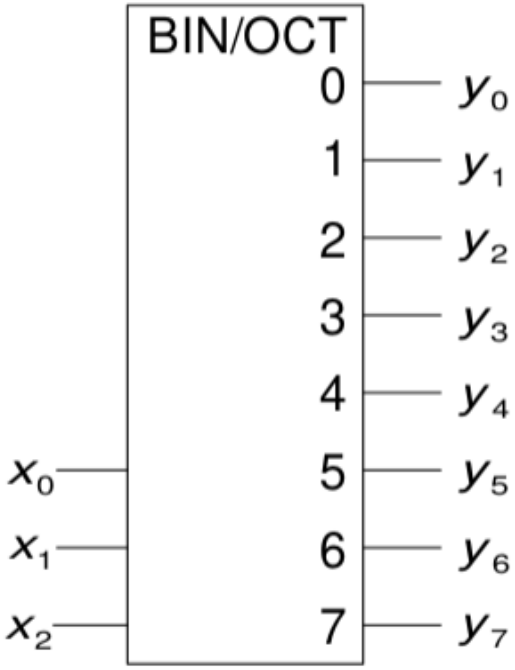
Dezimalzahl	Dualcode			1-aus-8-Code							
k	x_2	x_1	x_0	y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0
0	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
2	0	1	0	0	0	0	0	0	1	0	0
3	0	1	1	0	0	0	0	1	0	0	0
4	1	0	0	0	0	0	1	0	0	0	0
5	1	0	1	0	0	1	0	0	0	0	0
6	1	1	0	0	1	0	0	0	0	0	0
7	1	1	1	1	0	0	0	0	0	0	0

(One-Hot-)Decoderschaltnetz



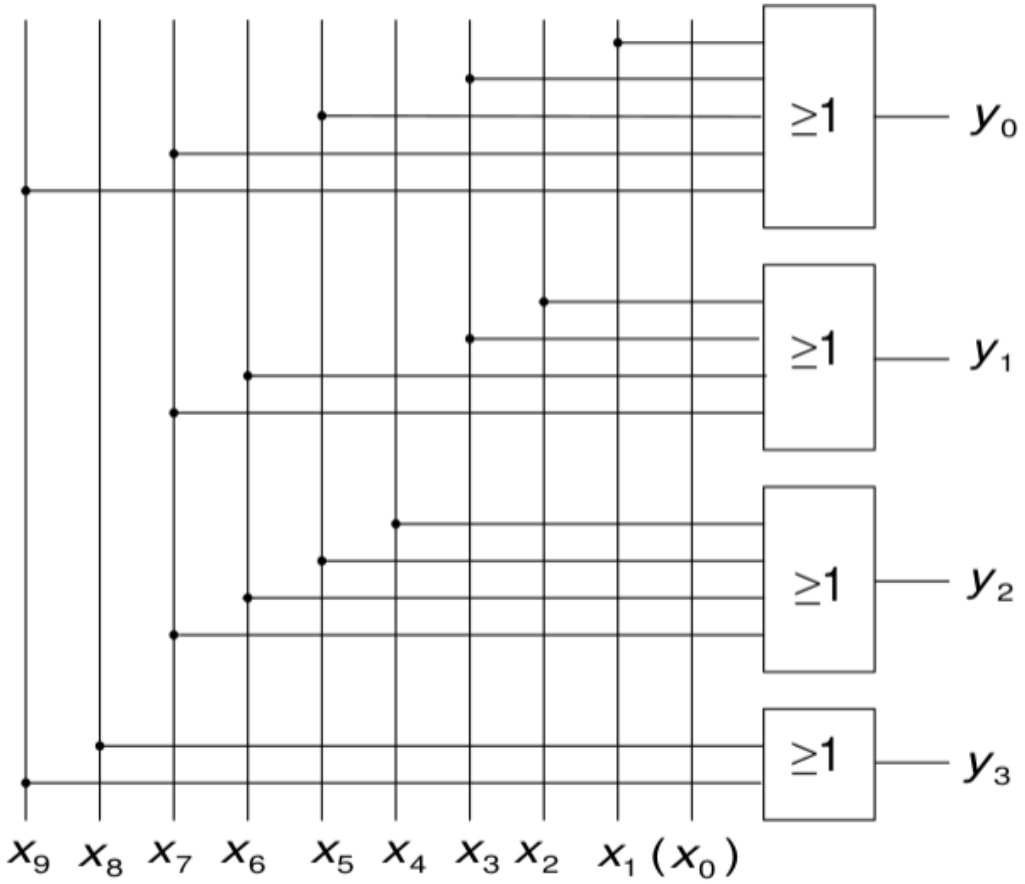
(One-Hot-)Decoderschaltnetz





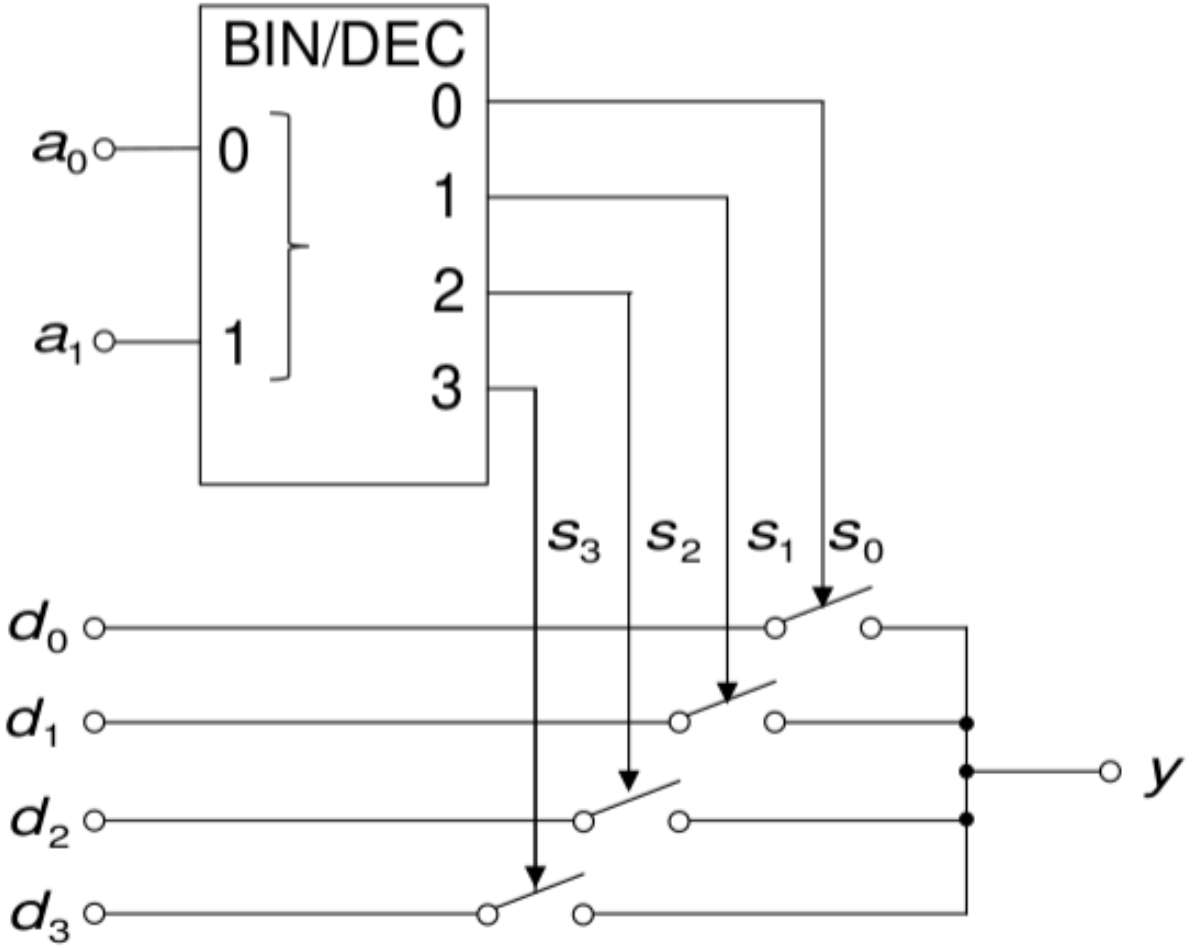
(One-Hot-)Encoderschaltnetz - Dezimal-Encoder

1-aus-10



Multiplexer

1-aus-4



Multiplexer

Wie sähe diese Funktion aus Software-Sicht aus?

Wie sehen die “Schalter“ in Hardware aus?

Multiplexer

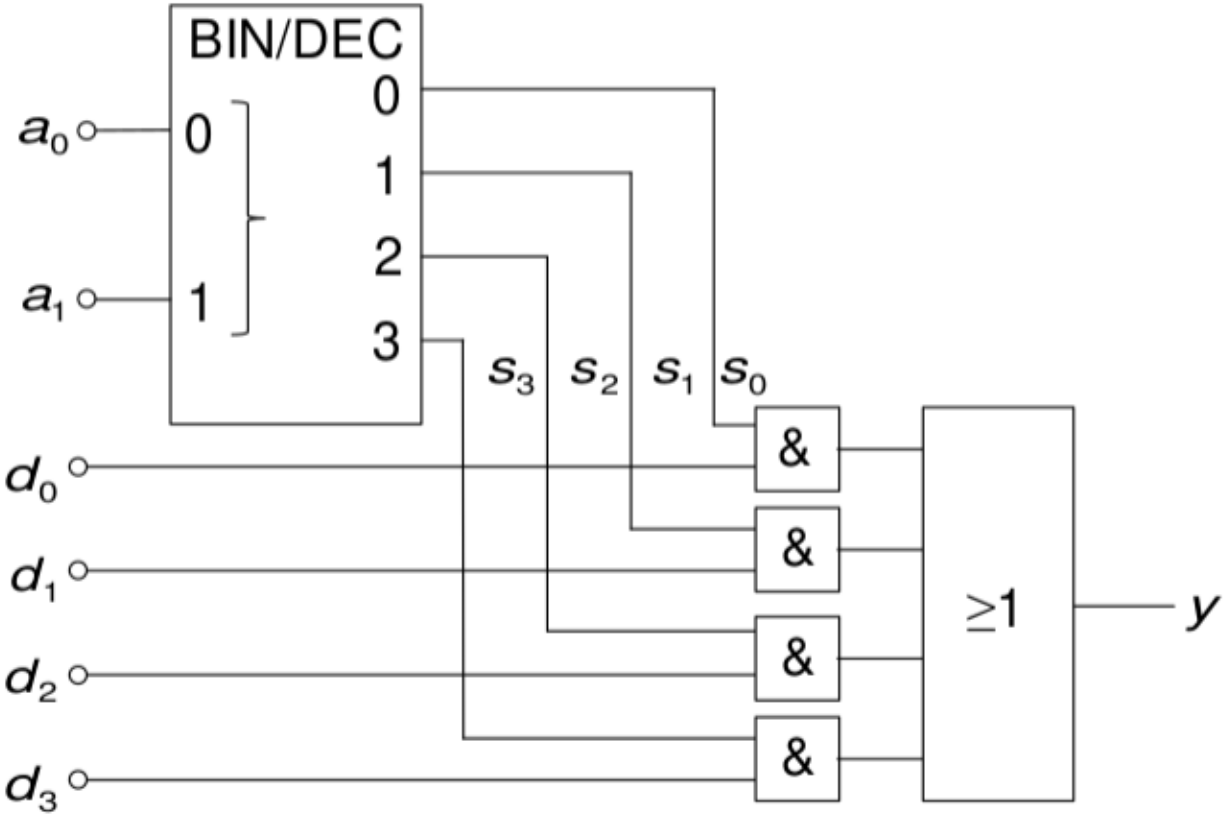
a_0	a_1	y
0	0	d_0
0	1	d_1
1	0	d_2
1	1	d_3

$$y = min_0 d_0 \vee min_1 d_1 \vee min_2 d_2 \vee min_3 d_3$$

$$y = \overline{a_1}\,\overline{a_0} d_0 \vee \overline{a_1} a_0 d_1 \vee a_1 \overline{a_0} d_2 \vee a_1 a_0 d_3$$

Multiplexer

Schalterrealisierung

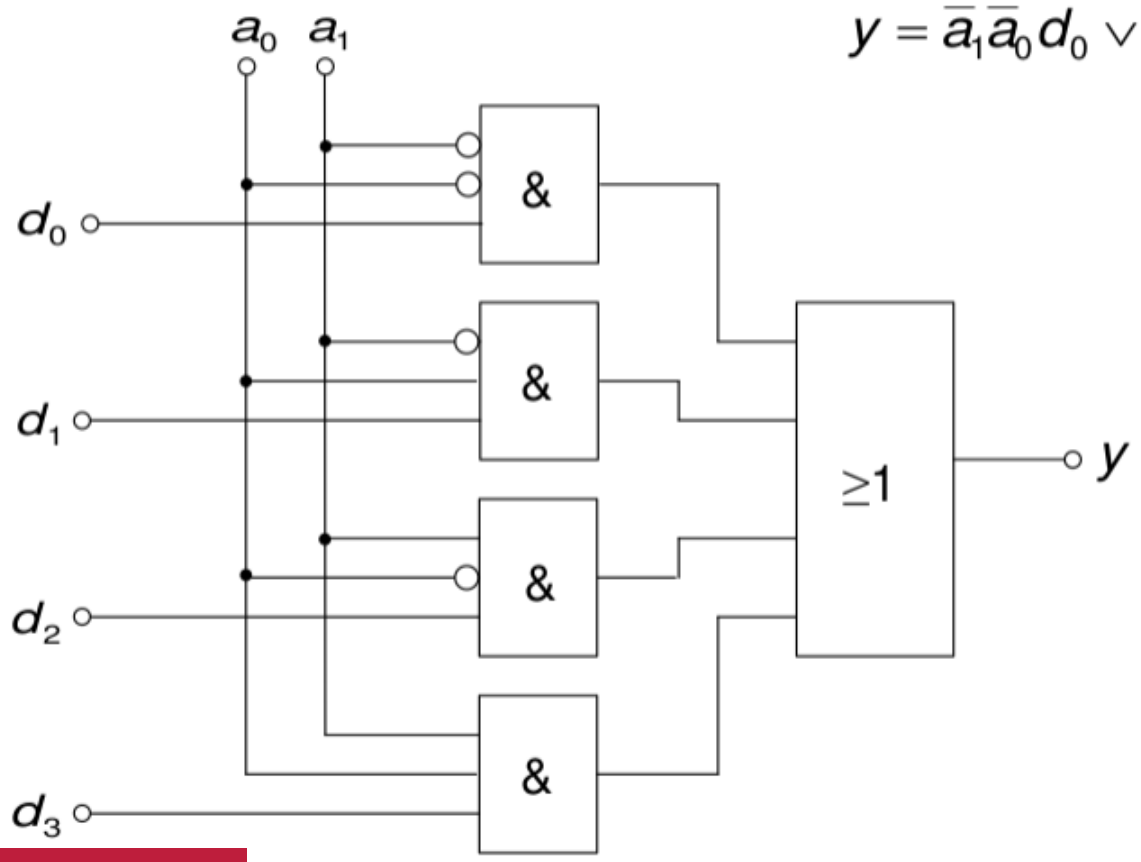


Multiplexer

Kombination Adress-Decoder und Schalter

$$s_i d_i = \min_i d_i$$

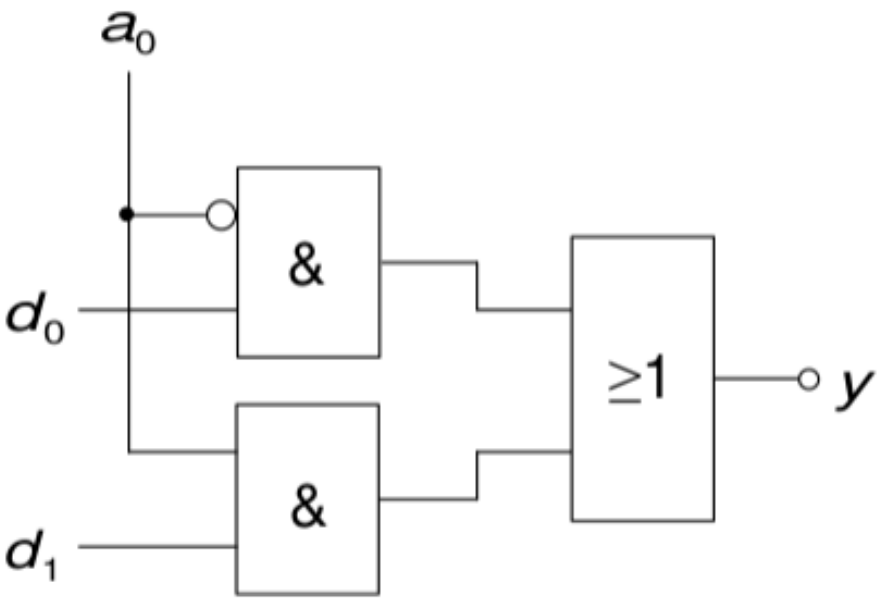
$$y = \bar{a}_1 \bar{a}_0 d_0 \vee \bar{a}_1 a_0 d_1 \vee a_1 \bar{a}_0 d_2 \vee a_1 a_0 d_3$$



2:1 MUX-Realisierung

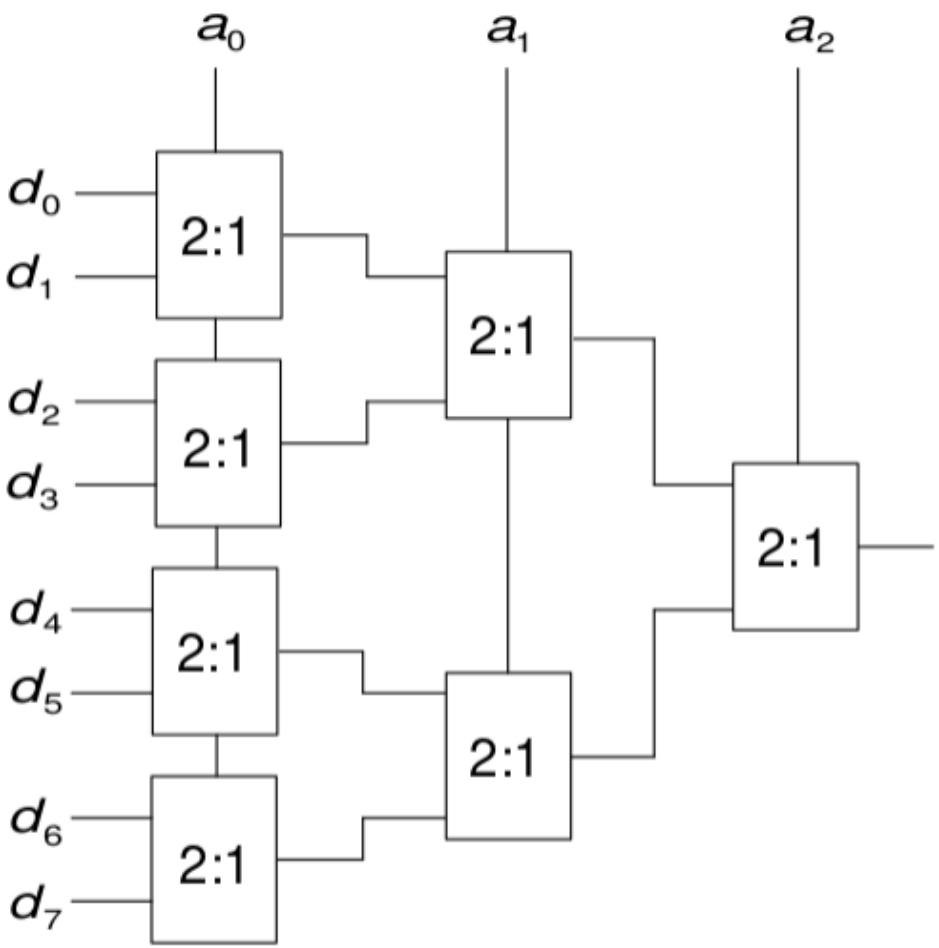
Kombination Adress-Decoder und Schalter

2:1 MUX



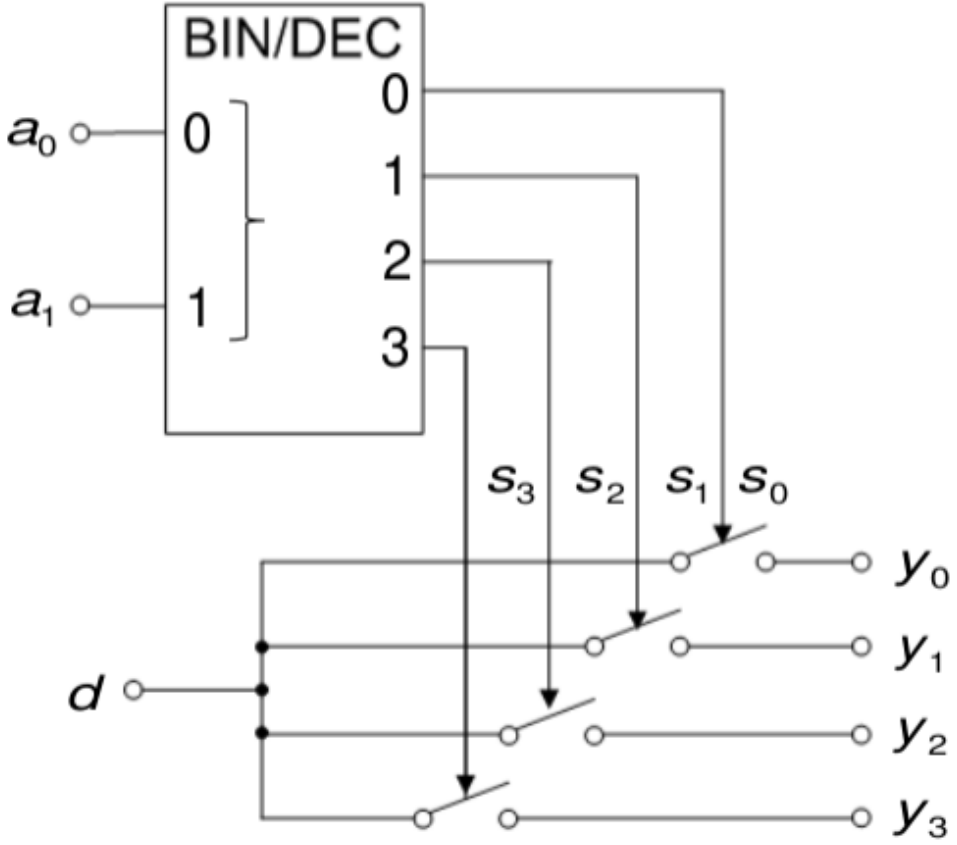
Hierarchische MUX-Realisierung

8:1 MUX als Hierarchie



Demultiplexer

1:4 Demultiplexer



Demultiplexer

a_0	a_1	y_3	y_2	y_1	y_0
0	0	0	0	0	d
0	1	0	0	d	0
1	0	0	d	0	0
1	1	d	0	0	0

$$y_0 = min_0 d = \overline{a_1}\ \overline{a_0} d$$

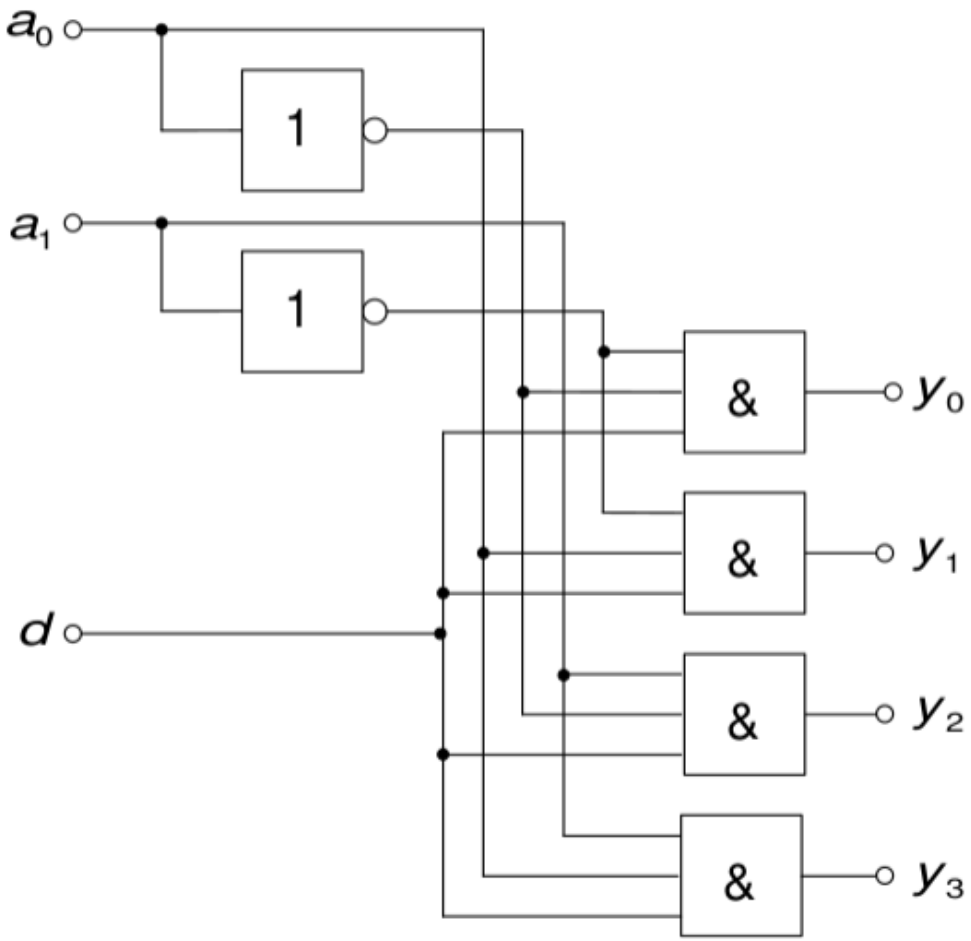
$$y_1 = min_1 d = \overline{a_1}\ a_0 d$$

$$y_2 = min_2 d = a_1\ \overline{a_0} d$$

$$y_3 = min_3 d = a_1\ a_0 d$$

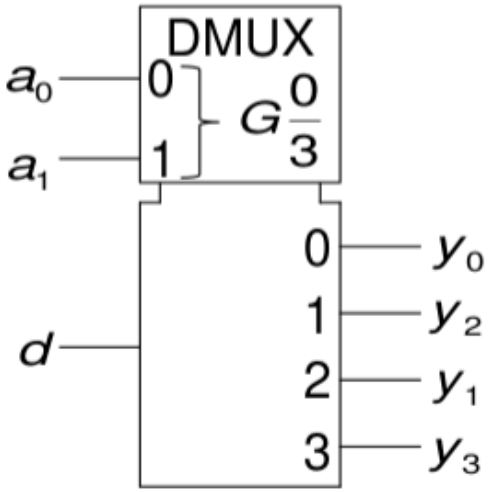
Demultiplexer

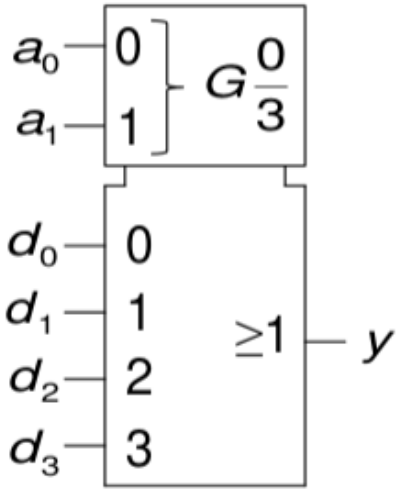
1:4 Demultiplexer



Multiplexer / Demultiplexer

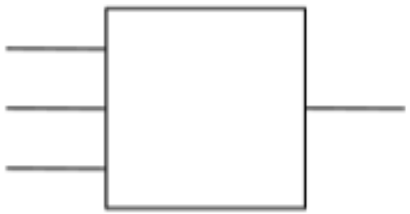
Schaltsymbole





Grundstruktur eines Speichers

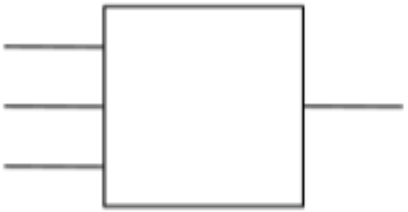
- Kombinatorische Logik



Output des Funktionsblocks basiert auf Eingangsbelegung

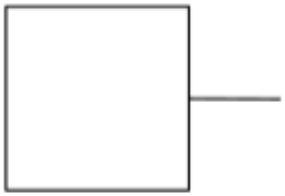
Grundstruktur eines Speichers

- Kombinatorische Logik



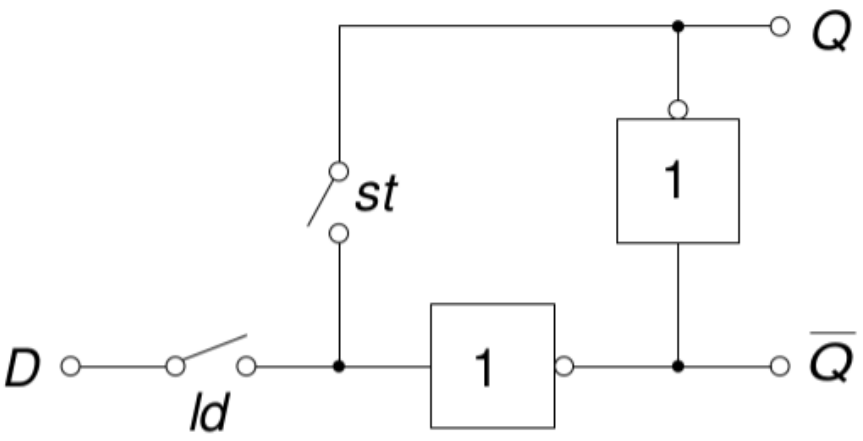
Output des Funktionsblocks basiert auf Eingangsbelegung

- Speicher



Output des Funktionsblocks unabhängig

Grundstruktur eines Speichers



ld = Load

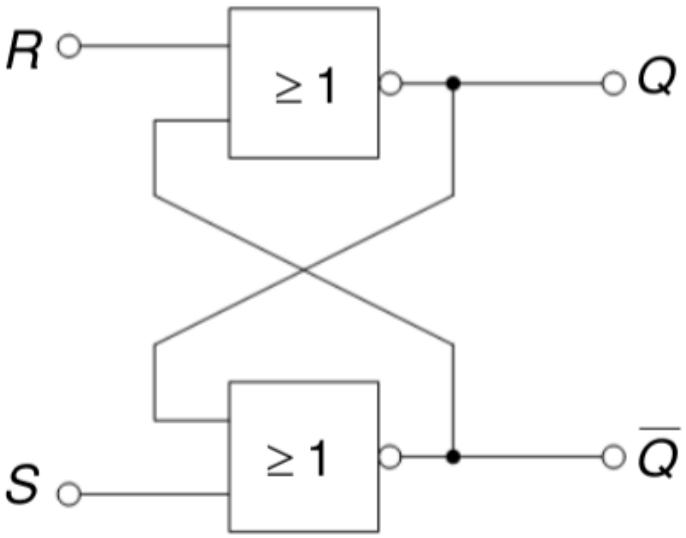
st = Store

Während des “Load“ ist $Q = D$

RS-Flip-Flop

- Flip-Flop: bistabile Kippschaltung

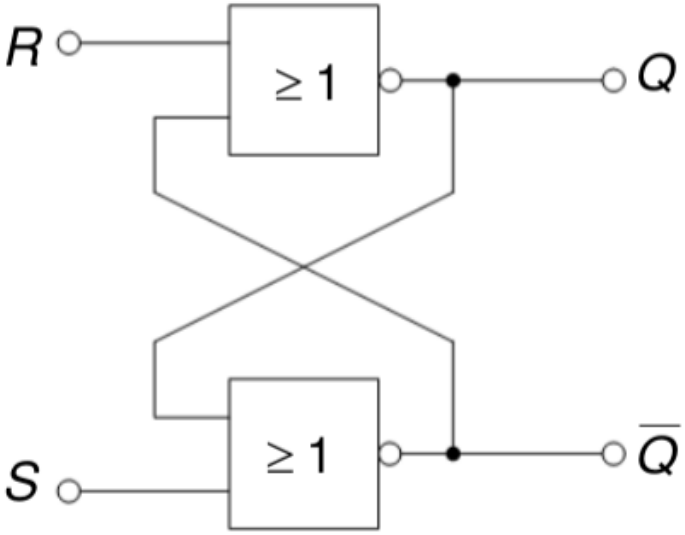
  RS-Flip-Flop

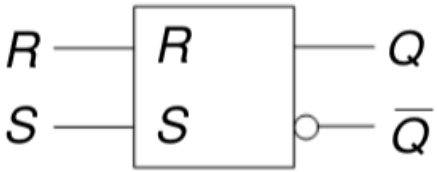


RS-Flip-Flop

- Flip-Flop: bistabile Kippschaltung

  RS-Flip-Flop





Funktionstabelle RS-Flip-Flop

S^n	R^n	Q_1^n	Q_2^n	Q_1^{n+1}	Q_2^{n+1}	Funktion
0	0	Q_1^n	$\overline{Q_1^n}$			
0	1	Q_1^n	$\overline{Q_1^n}$			
1	0	Q_1^n	$\overline{Q_1^n}$			
1	1	Q_1^n	$\overline{Q_1^n}$			

Funktionstabelle RS-Flip-Flop

S^n	R^n	Q_1^n	Q_2^n	Q_1^{n+1}	Q_2^{n+1}	Funktion
0	0	Q_1^n	$\overline{Q_1^n}$	Q_1^n	$\overline{Q_1^n}$	Speichern
0	1	Q_1^n	$\overline{Q_1^n}$	0	1	Zurücksetzen
1	0	Q_1^n	$\overline{Q_1^n}$	1	0	Setzen
1	1	Q_1^n	$\overline{Q_1^n}$	(0)	(0)	“Verboten“

D-Flip-Flop

Delay-Flip-Flop

D^n	Q^{n+1}	Funktion
0		
1		

D-Flip-Flop

Delay-Flip-Flop

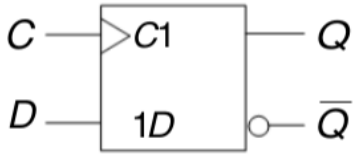
D^n	Q^{n+1}	Funktion
0	0	Übernehmen (rücksetzen)
1	1	Übernehmen (setzen)

Taktung von Flip-Flops

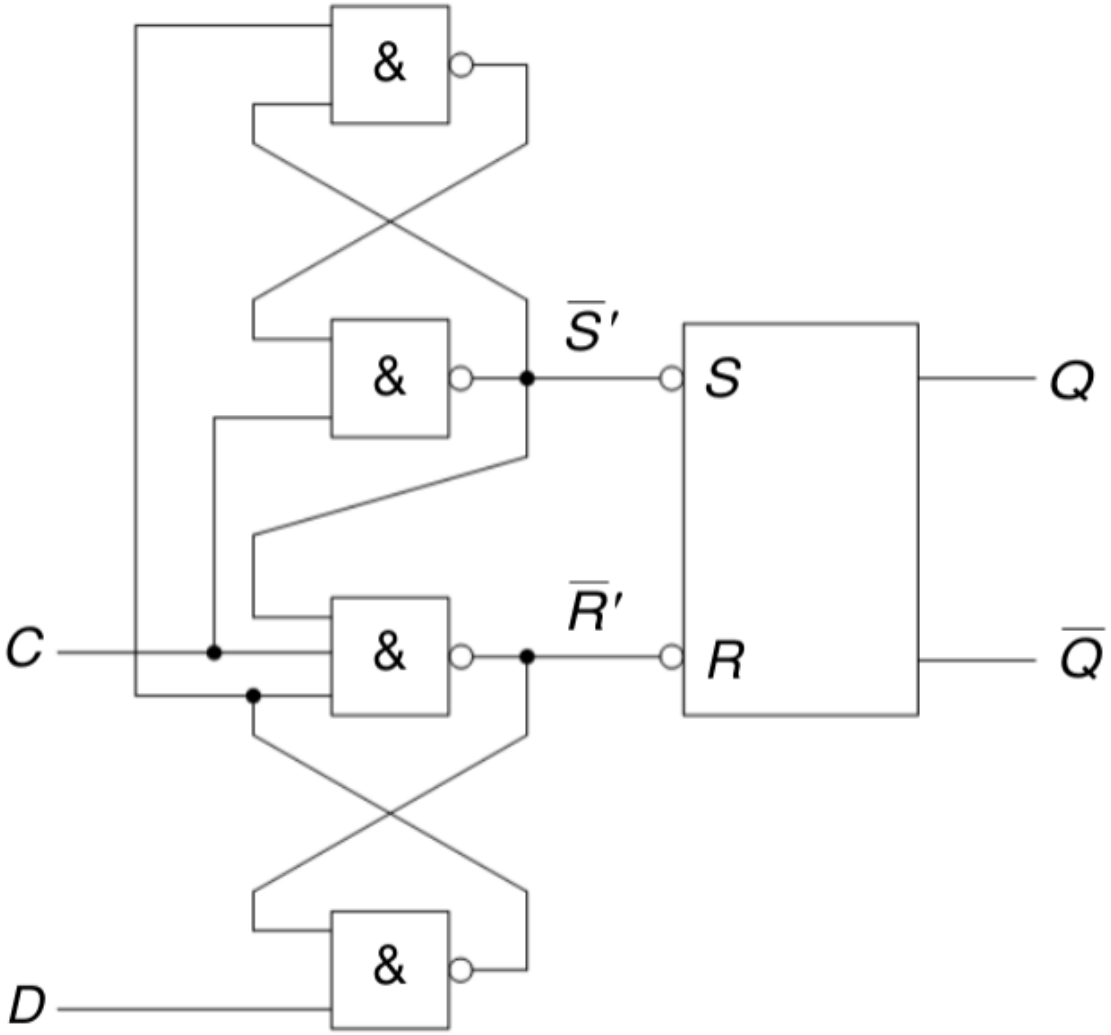
- Aktivierung der Flip-Flops
- Taktsteuerung
  - Taktzustand (High oder Low)
  - Taktänderung (Taktflanke)

Taktung von Flip-Flops

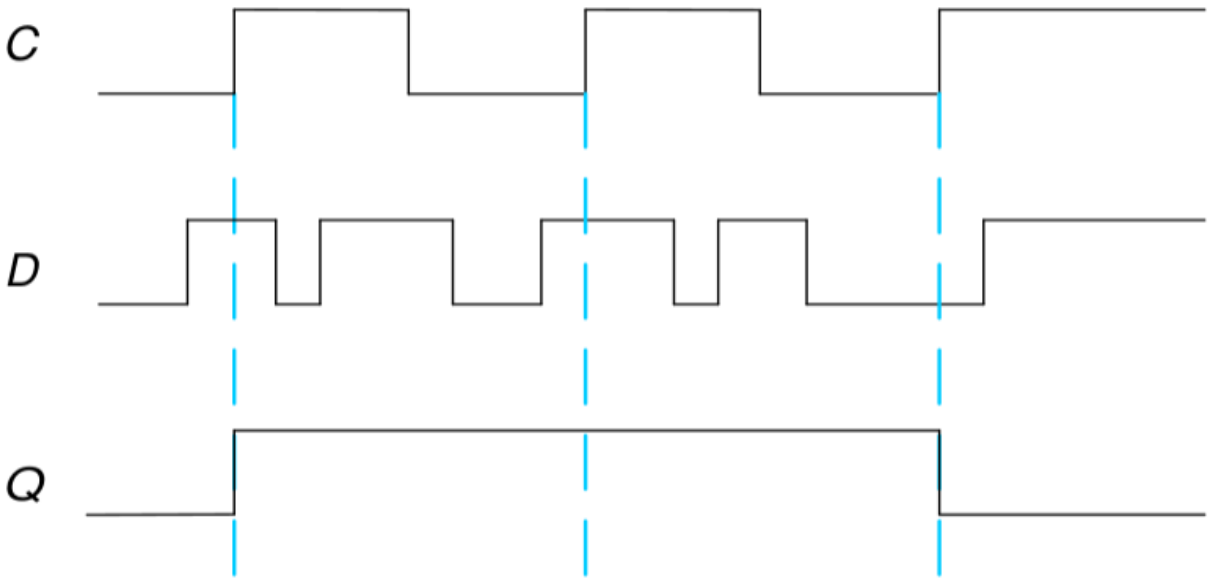
- Aktivierung der Flip-Flops
- Taktsteuerung
  - Taktzustand (High oder Low)
  - Taktänderung (Taktflanke)



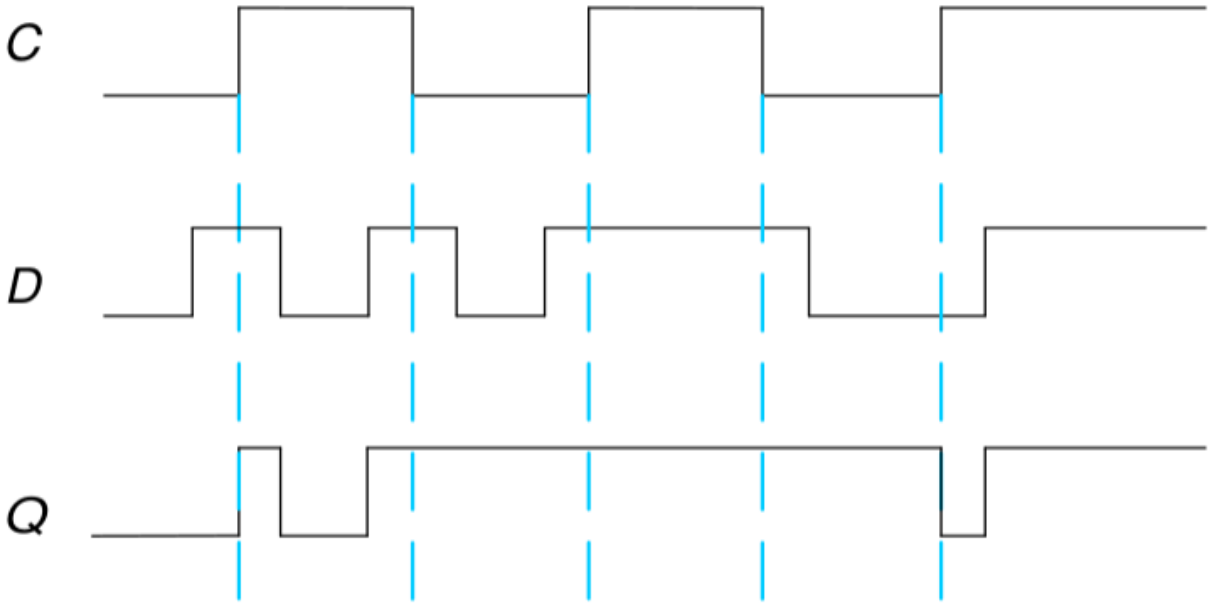
Taktflankegetriggertes D-Flip-Flop



Taktflankegetriggertes D-Flip-Flop

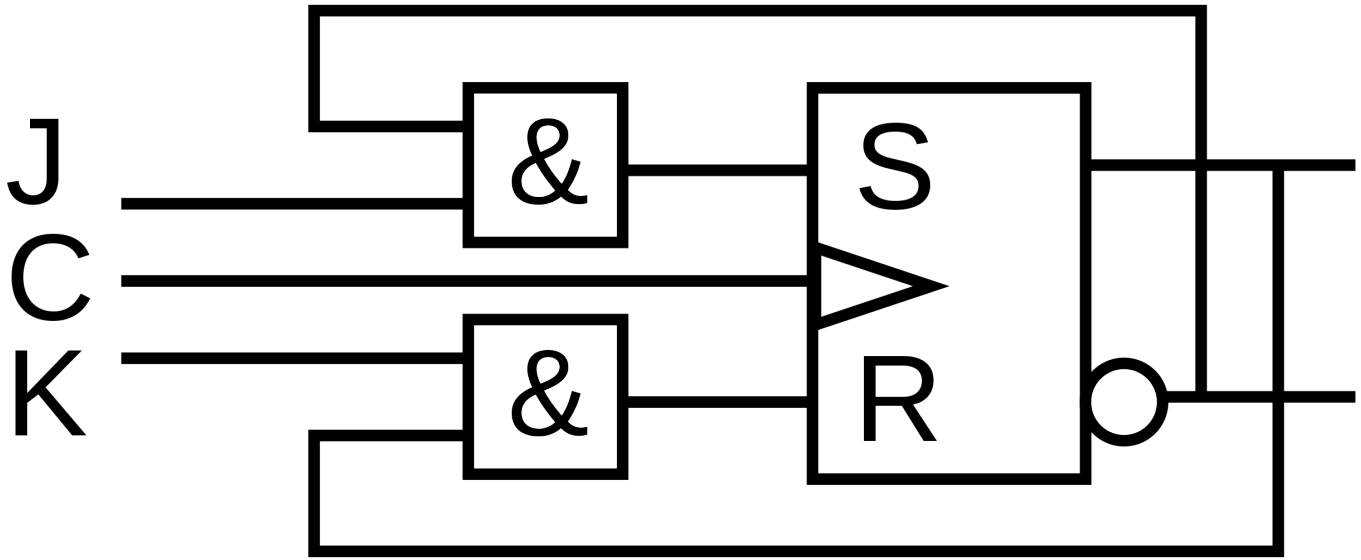


Taktpegelgesteuertes D-Flip-Flop / D-Latch



JK-Flip-Flop

Jump-Kill-Flip-Flop

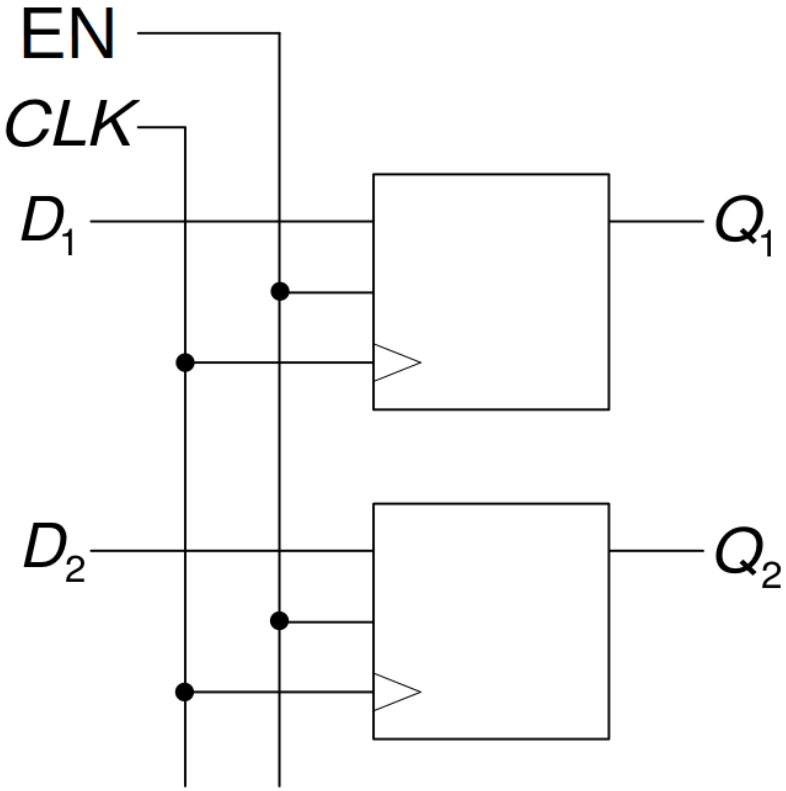


JK-Flip-Flop

S^n	R^n	Q_1^n	Q_2^n	Q_1^{n+1}	Q_2^{n+1}	Funktion
0	0	Q_1^n	$\overline{Q_1^n}$	Q_1^n	$\overline{Q_1^n}$	Speichern
0	1	Q_1^n	$\overline{Q_1^n}$	0	1	Zurücksetzen
1	0	Q_1^n	$\overline{Q_1^n}$	1	0	Setzen
1	1	Q_1^n	$\overline{Q_1^n}$	X	X	**Toggle**

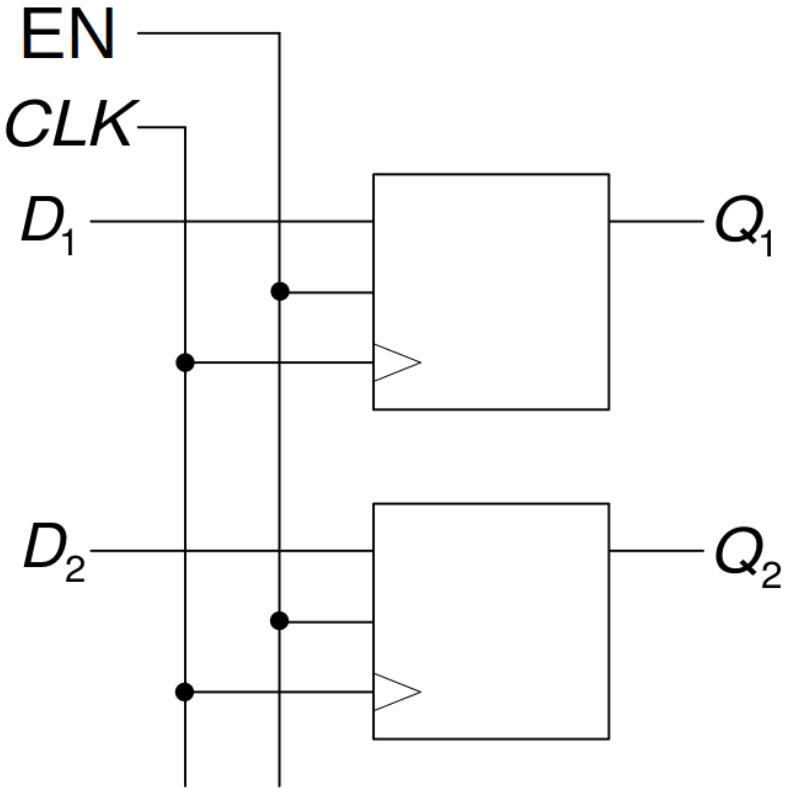
Register

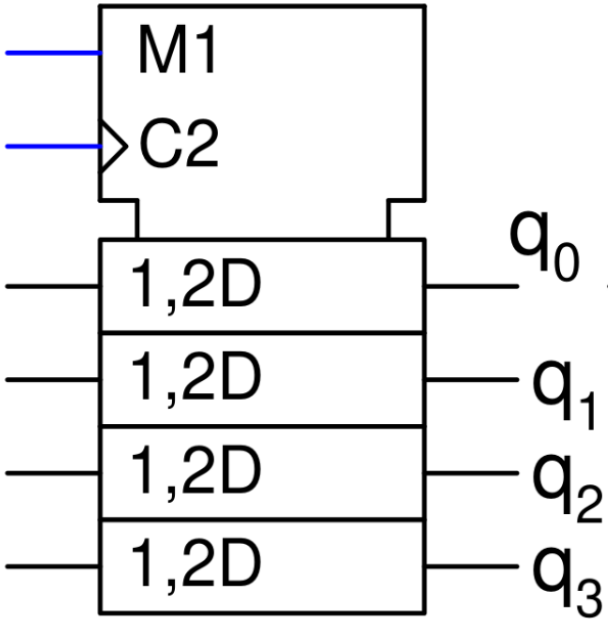
- n parallele D-Flip-Flops



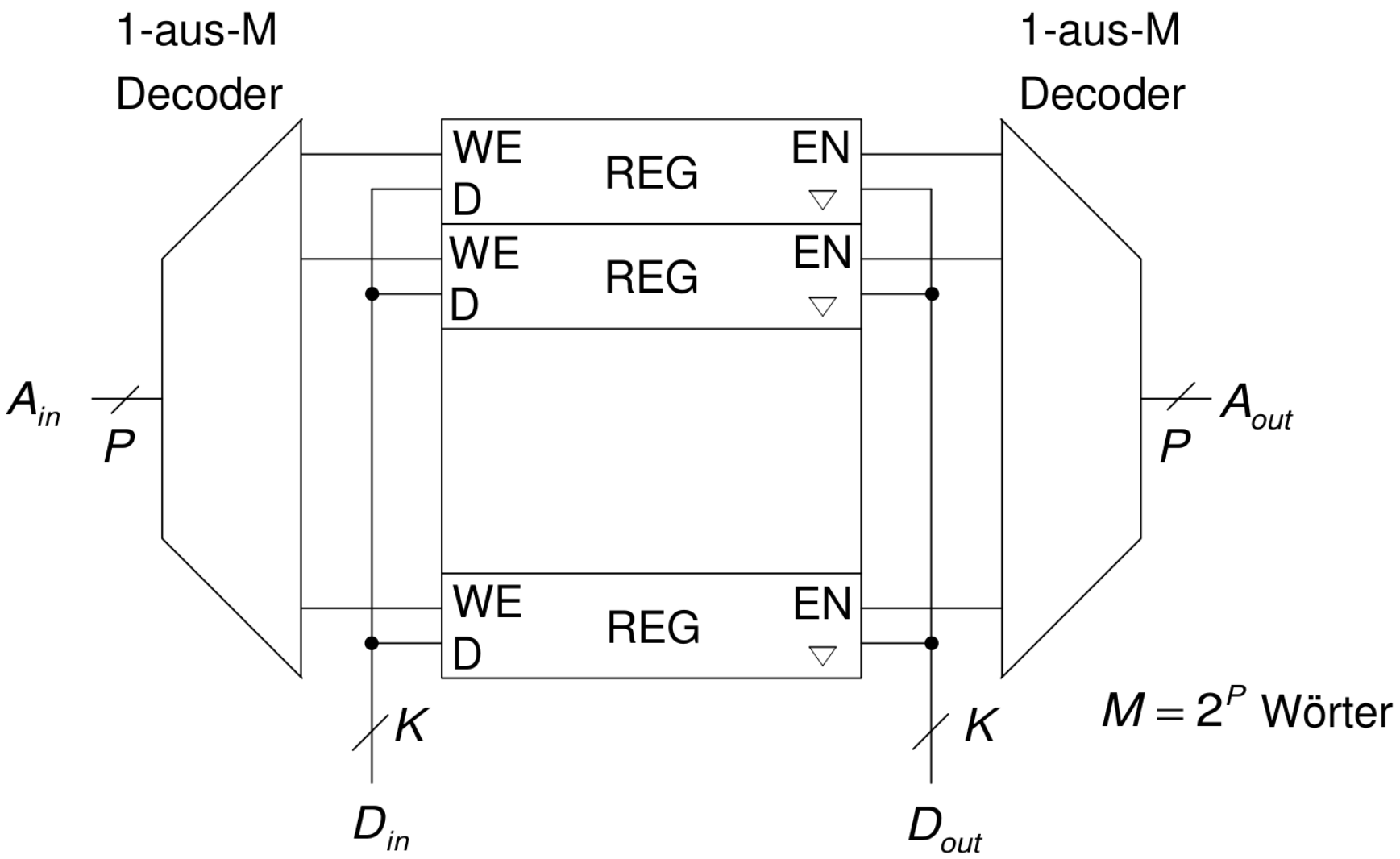
Register

- n parallele D-Flip-Flops



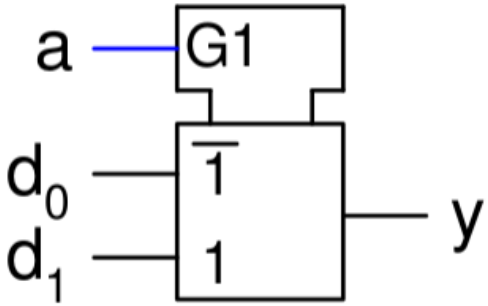


Register-File



Übungen MUX

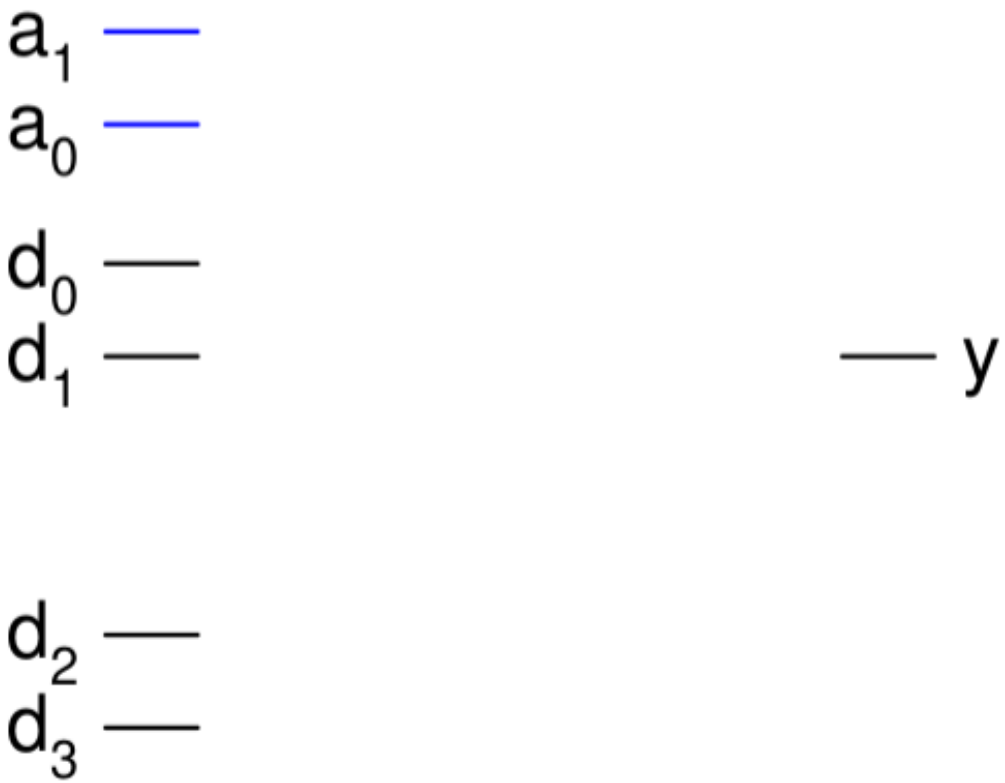
a) Beschreiben Sie die Funktionsweise des dargestellten 2:1 Multiplexers



a	y
0	
1	

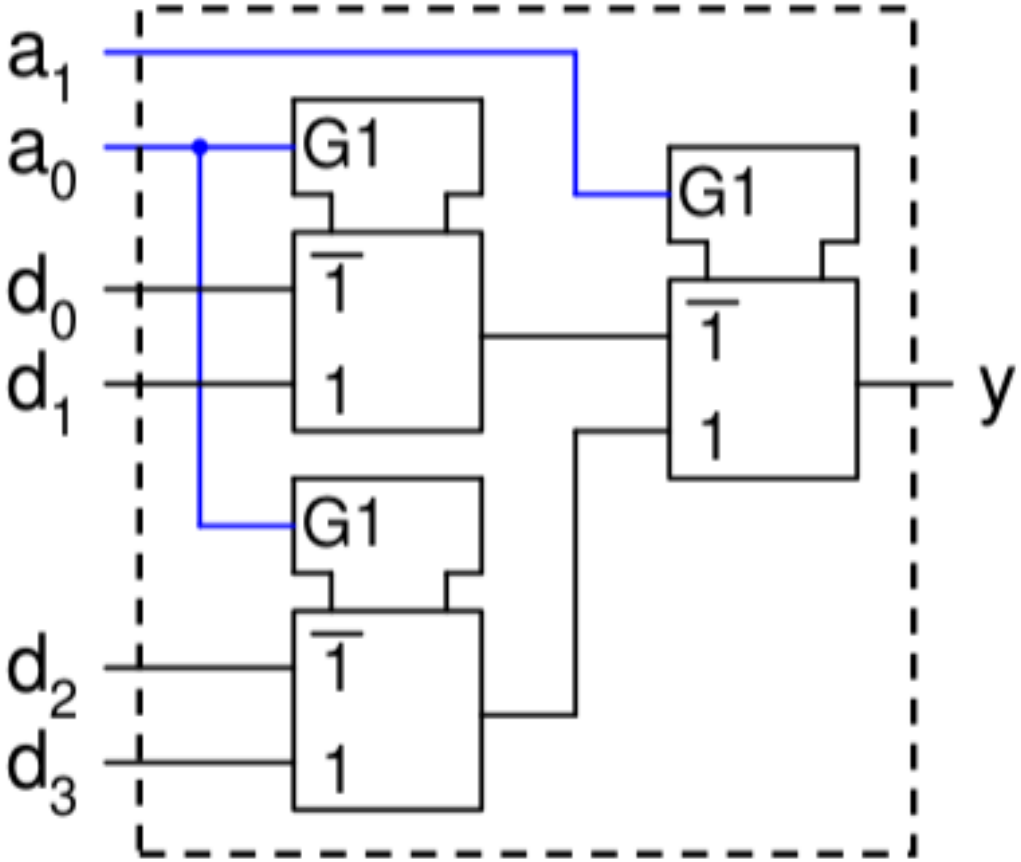
Übungen MUX

b) Erzeugen Sie aus der Verschaltung mehrerer 2:1 Multiplexer einen 4:1 Multiplexer



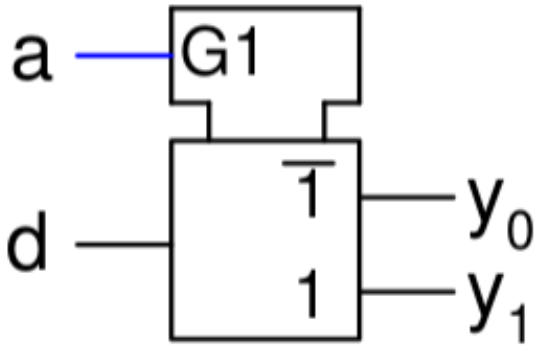
Übungen MUX

b) Erzeugen Sie aus der Verschaltung mehrerer 2:1 Multiplexer einen 4:1 Multiplexer



Übungen DEMUX

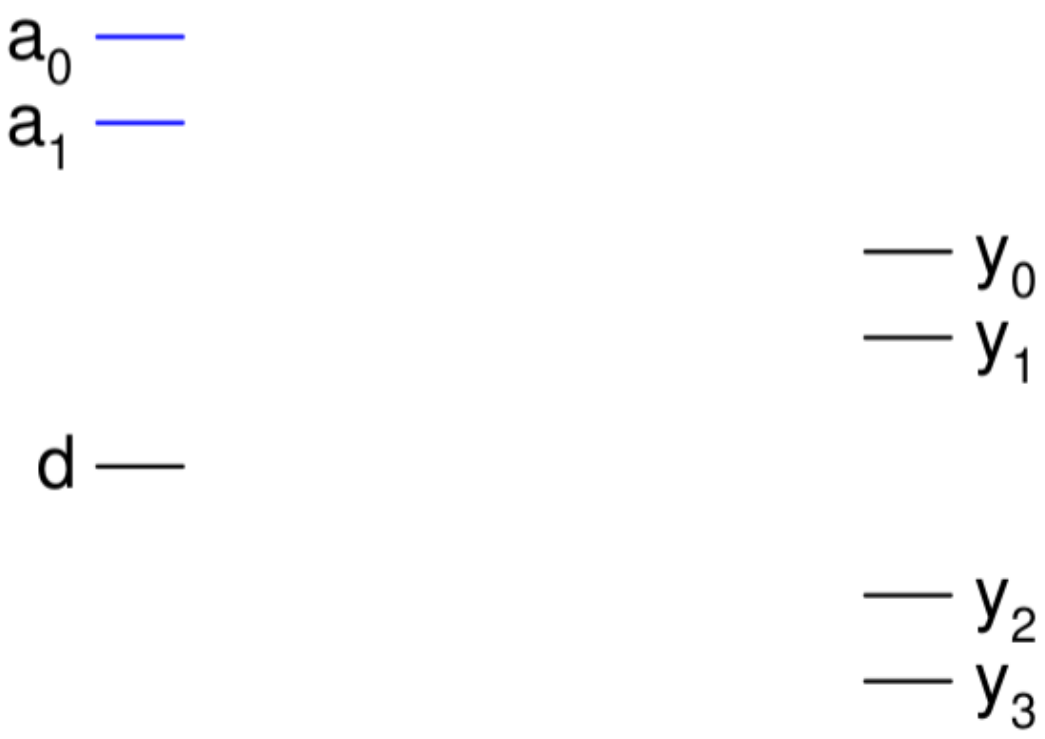
a) Beschreiben Sie die Funktionsweise des dargestellten 1:2 Demultiplexers



a	d	y_0	y_1
0	0		
0	1		
1	0		
1	1		

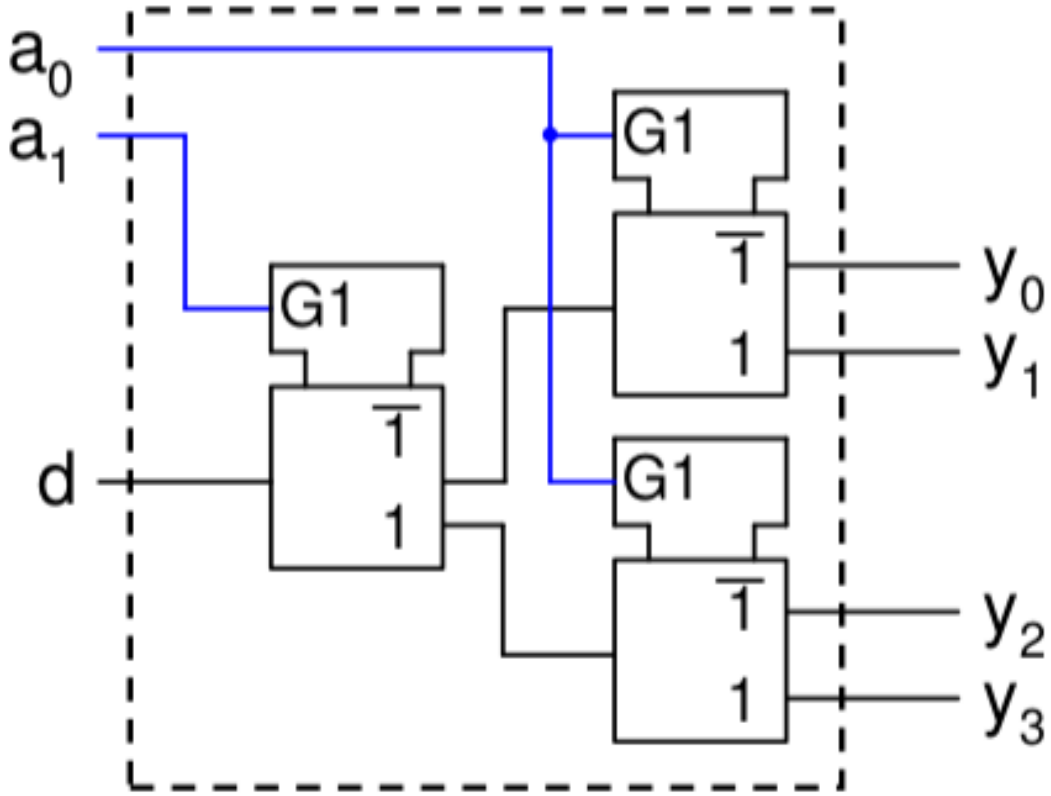
Übungen DEMUX

b) Erzeugen Sie aus der Verschaltung mehrerer 1:2 Demultiplexer einen 1:4 Demultiplexer



Übungen DEMUX

b) Erzeugen Sie aus der Verschaltung mehrerer 1:2 Demultiplexer einen 1:4 Demultiplexer



Übungen MUX-Schaltung

Es soll ein 4-Bit-Hardware-Shifter entworfen werden, der in der Lage ist, eine 4-Bit-Zahl $x = x_3x_2x_1x_0$ um 0-3 Stellen nach links zu schieben (Ausgangssignal $y = y_3y_2y_1y_0$).

a) Stellen Sie die Funktionstabelle auf

b) Mit welcher mathematischen Operation y=f(x) kann ein Links-Shift auch beschrieben werden?

L-Shift	$y = y_3y_2y_1y_0$
0-bit	
1-bit	
2-bit	
3-bit	

Übungen MUX-Schaltung

Es soll ein 4-Bit-Hardware-Shifter entworfen werden, der in der Lage ist, eine 4-Bit-Zahl $x = x_3x_2x_1x_0$ um 0-3 Stellen nach links zu schieben (Ausgangssignal $y = y_3y_2y_1y_0$).

a) Stellen Sie die Funktionstabelle auf

b) Mit welcher mathematischen Operation y=f(x) kann ein Links-Shift auch beschrieben werden?

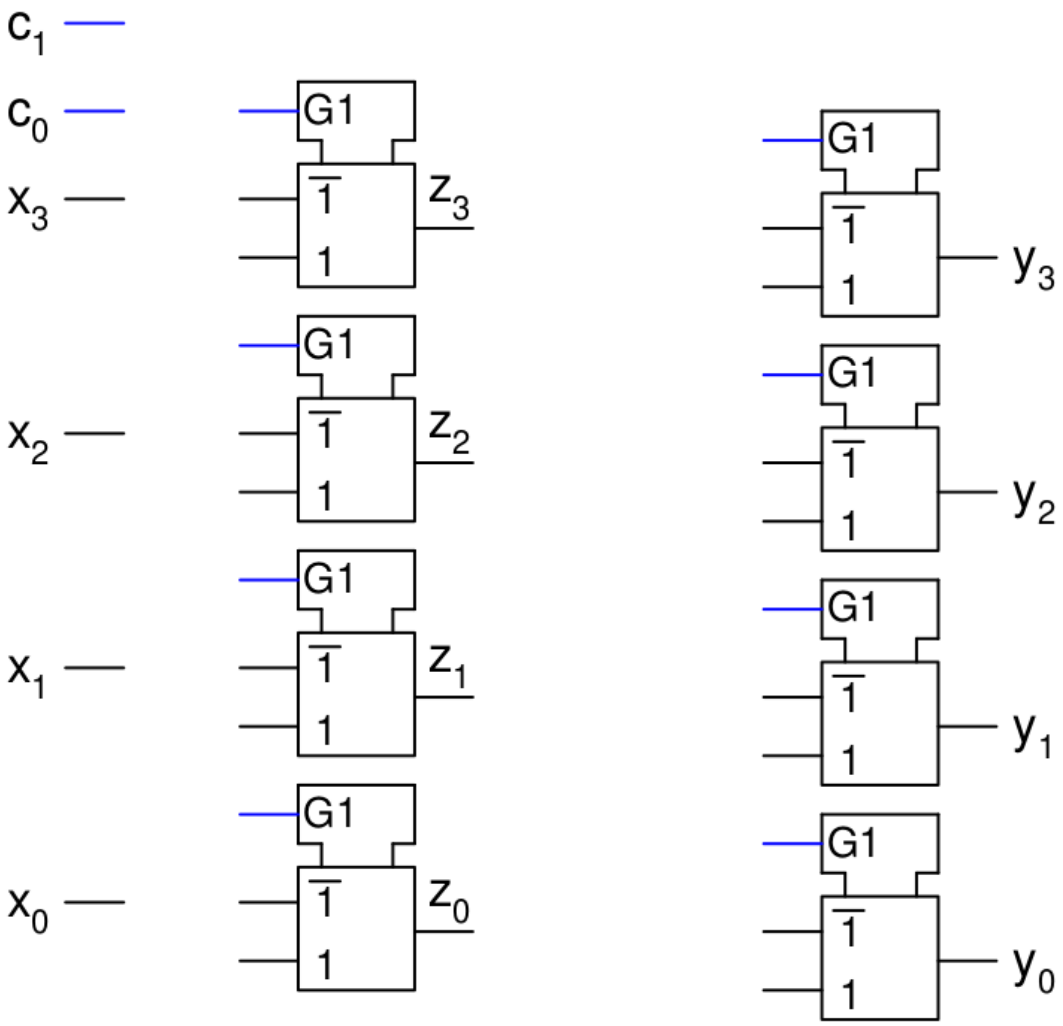
L-Shift	$y = y_3y_2y_1y_0$
0-bit	$x_3x_2x_1x_0$
1-bit	$x_2x_1x_00$
2-bit	x_1x_000
3-bit	x_0000

Links-Shift entspricht einer Multiplikation mit 2 –> $y = x \cdot 2^m$

Übungen MUX-Schaltung

c) Das 2-Bit-Signal $c = c_1 c_0$ beschreibt als Dualzahl die Shift-Weite. Konstruieren Sie zunächst aus 4 2:1-Multiplexern eine Architektur, die in der Lage ist, Links-Shifts der Weiten 0 und 1 durchzuführen. Das 4-bit-Ausgangssignal dieser Stufe sei $z = z_3 z_2 z_1 z_0$.

Übungen MUX-Schaltung



Übungen MUX-Schaltung

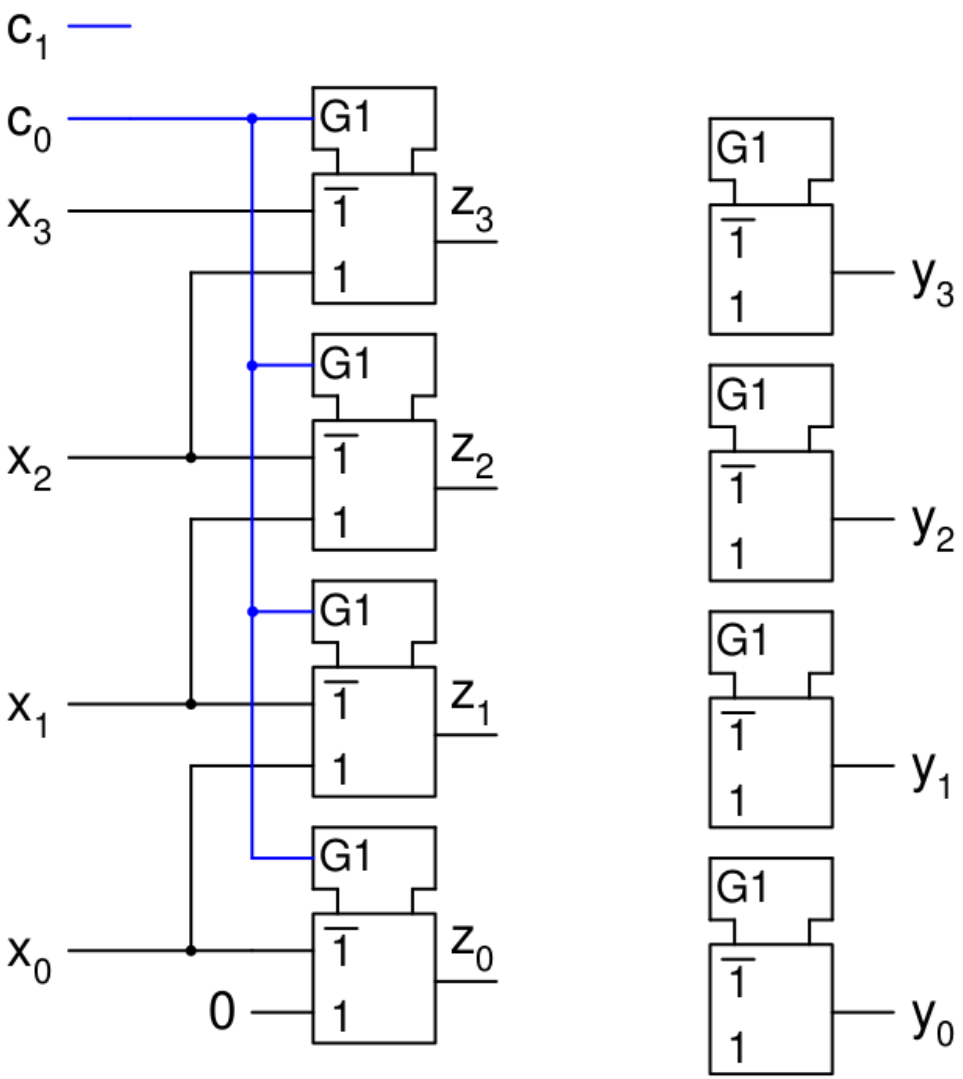
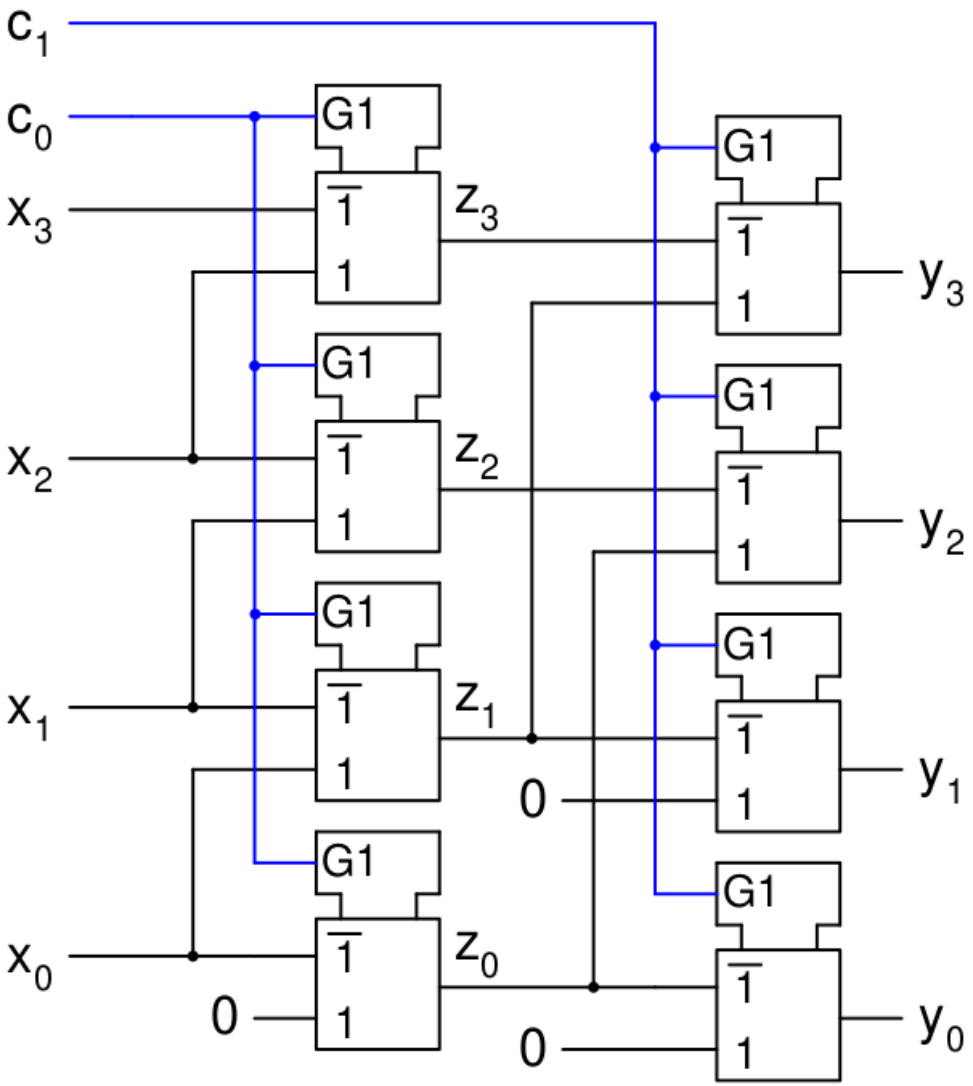


Abbildung 1:

Übungen MUX-Schaltung

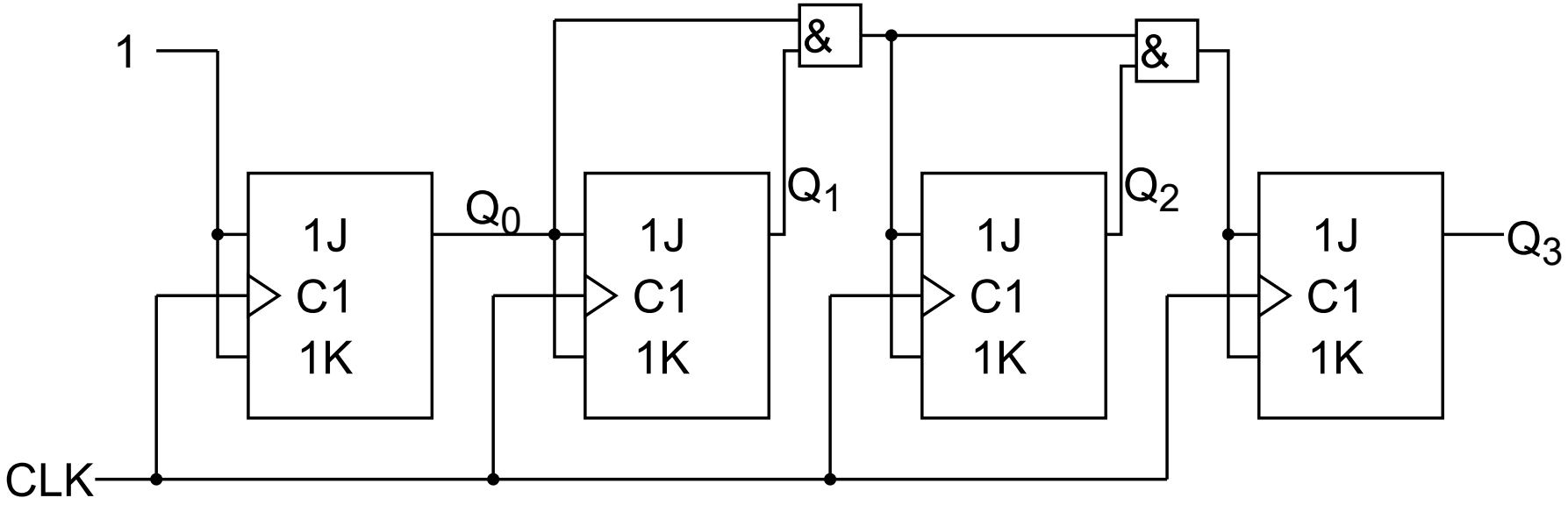
d) Erweitern Sie die ermittelte Architektur, so dass Shift-Weiten von 0-3 bit möglich sind.

Übungen MUX-Schaltung



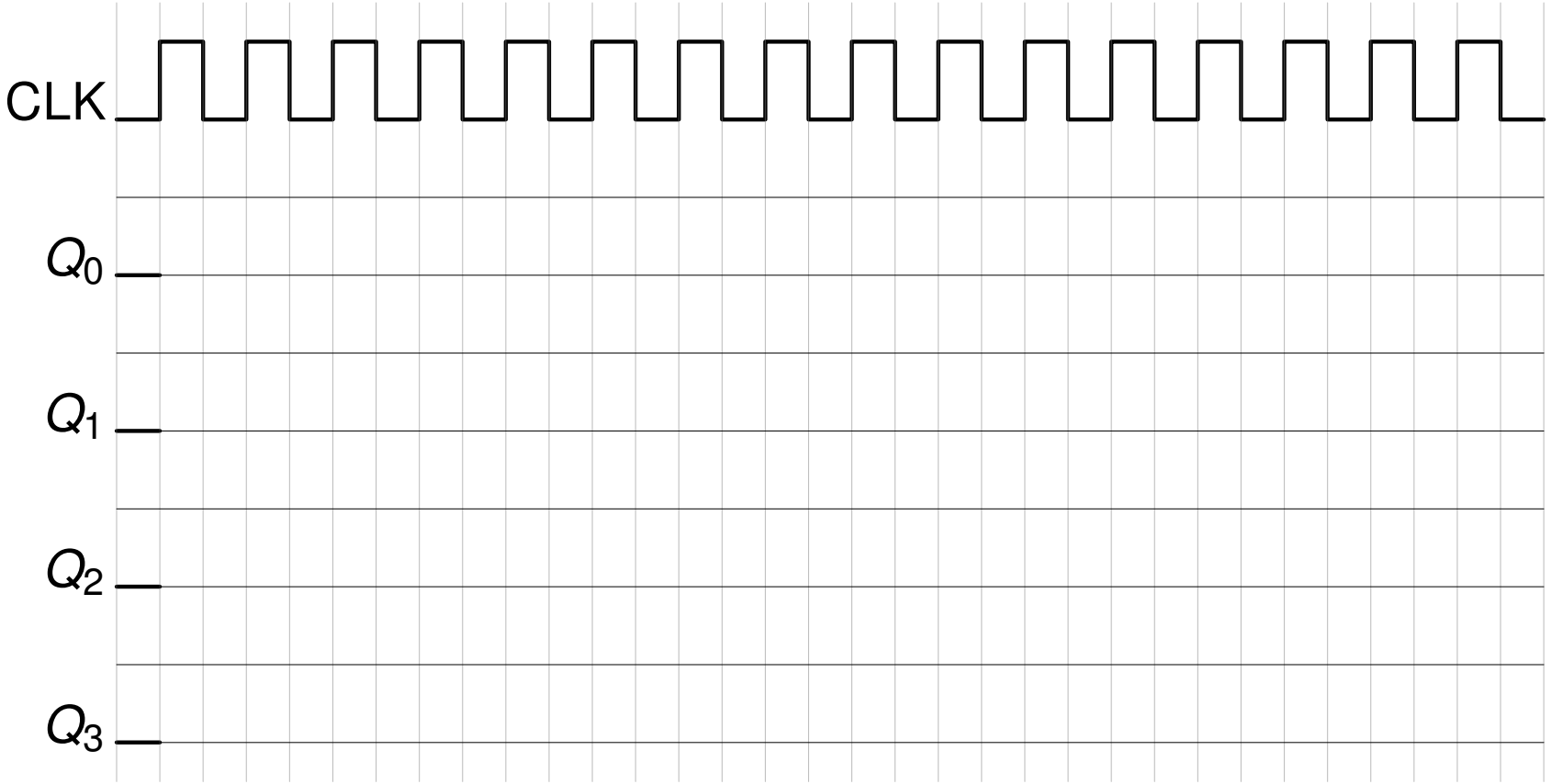
Übungen Flip-Flop Timing

Die folgende Schaltung aus J-K-Flip-Flops realisiert eine Funktion mit CLK als Eingang und Q_0, Q_1, Q_2 und Q_3 als Ausgänge.



Übungen Flip-Flop Timing

Vervollständigen Sie das Timing-Diagramm. Q_i starten jeweils mit 0.



Übungen Flip-Flop Timing

Interpretieren Sie die von der Schaltung realisierte Funktion.

EDA-Playground

- Codebeispiele in Vorlesung und Übung
- Kostenloses Onlinetool
- Registrierung mit Uni-Mailadresse erforderlich
- Benutzung von Industriestandard-Simulationstools
- Anzeige von Signalen als Waveform
- Ausführen von Tests

GIT-Übungen

- Übungen für zuhause und zur Klausurvorbereitung
- Automatische Überprüfung der Lösung
- Bereitstellung für alle, die in StudIP an der Veranstaltung teilnehmen **und** sich beim GIT des GITZ (`https://git.rz.tu-bs.de`) bis Dienstag einmal *mit dem Studierendenaccount* angemeldet haben